GAMES101 Assignment2 - Triangles and Z-buffering

Implemented Features

Triangle Rasterization

Implemented triangle rasterization by:

- Computing the triangle bounding box
- Iterating through all pixels inside the bounding box
- Testing whether the pixel center is inside the triangle using cross products

Inside Triangle Test

Implemented `insideTriangle()` using cross product direction tests.

The algorithm checks whether a sample point lies on the same side of all triangle edges.

Z-buffer

Implemented depth testing using a depth buffer.

For each pixel:

- Interpolated depth is computed using barycentric coordinates
- The interpolated depth is compared with the current depth buffer
- The pixel is updated only if the current triangle is closer to the camera

Barycentric Interpolation

Used barycentric coordinates to interpolate the depth value across the triangle surface.

Bonus: 2x2 Super Sampling Anti-Aliasing (SSAA)

Implemented 2×2 SSAA anti-aliasing.

For each pixel:

- 4 sub-samples are evaluated
- Each sample maintains its own depth value
- Final pixel color is computed by averaging all sub-sample colors

This significantly reduces jagged edges along triangle boundaries.

Technologies Used

- C++
- Eigen
- OpenCV
- CMake

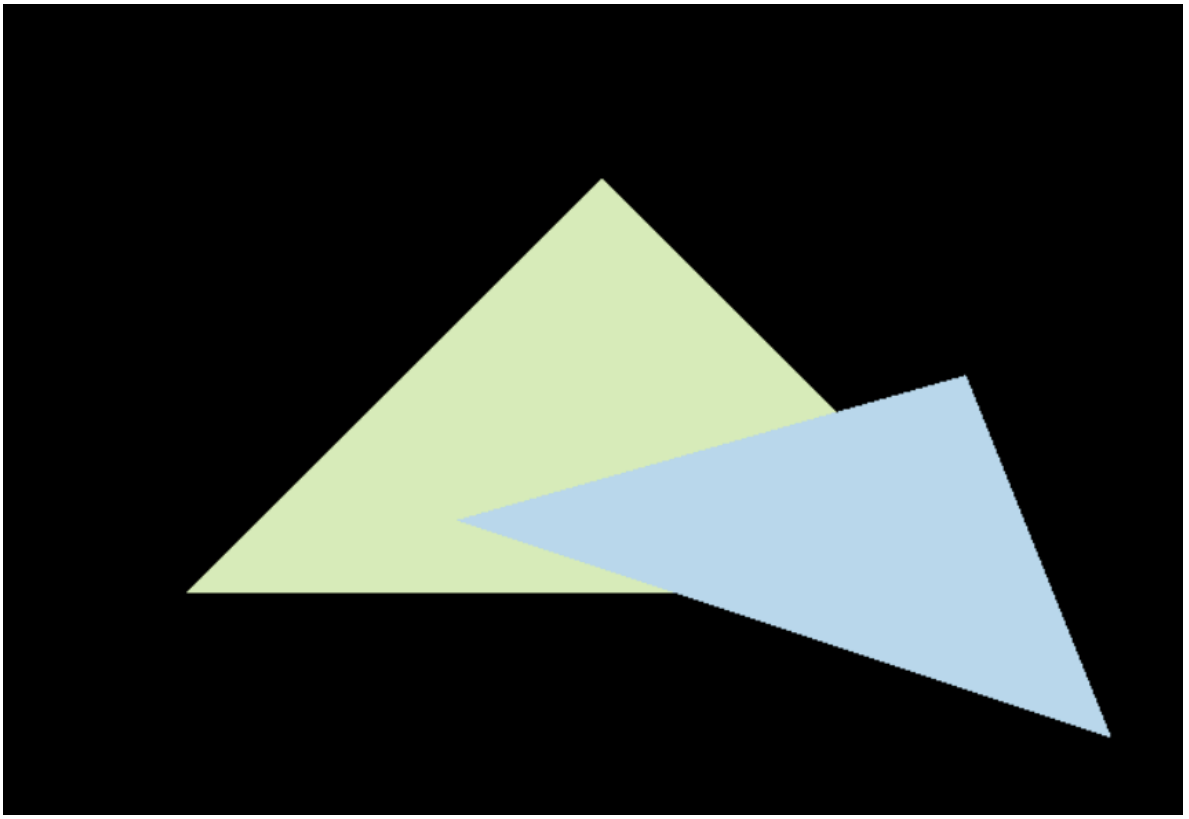
Result

Successfully rendered filled triangles with:

- correct depth testing
- triangle rasterization
- anti-aliasing using SSAA

Without SSAA

The triangle edges show visible jagged artifacts due to single-sample rasterization.



With 2x2 SSAA

The triangle edges become significantly smoother by averaging four sub-samples per pixel.